Containerized Microservices Data Collection and Analytics System

Prepared by: Mahmoud Haifawi

March-2023

1. Abstract

This system is designed with efficiency and scalability in mind, leveraging a comprehensive suite of Docker files to provide robust containerization for each individual service. I have also implemented a masterful Docker Compose file that ensures seamless and streamlined orchestration of the entire system.

One of the primary goals of our project is to provide authorized users with a secure and intuitive platform for data collection.

The EnterDataWebApp is a prime example of this, offering a simple and accessible way for users to submit their ages to the system.

Another key feature of our system is the ability to process and analyze data in real-time, allowing for the extraction of valuable insights and trends.

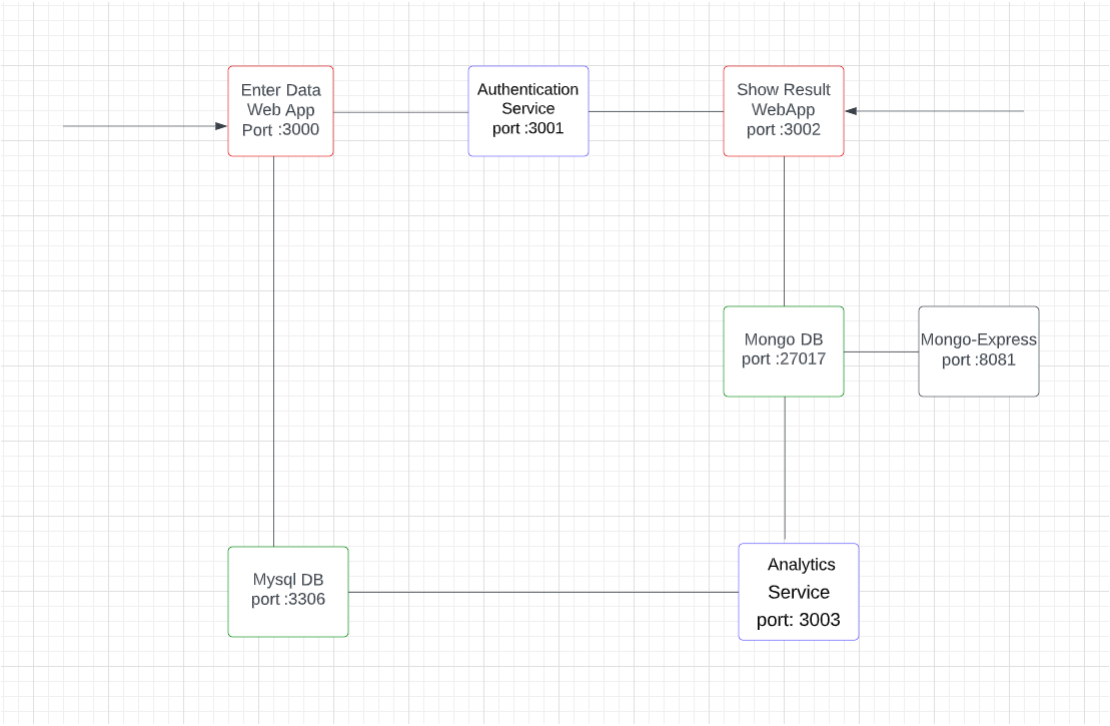
The ShowResultWebApp is a prime example of this, providing users with an easy-to-use interface that displays the average ages of all submitted data.

Through a combination of cutting-edge containerization techniques and streamlined data analytics, our system represents the pinnacle of modern data management and analysis.

2. System Architecture

Overview My project is basically a system that get ages from authorized users , using EnterDataWebApp , and shows the average of ages using ShowResultWebApp .

The figure below demonstrates the structure of the whole system:



3. Enter Data Web Application

The Enter Data Web Application is a secure and user-friendly web application that runs on port 3000. Its primary purpose is to collect age data from authorized users, ensuring that only authenticated individuals are granted access to the system.

```
enterdata:
  container_name: enterdata
  build: ./EnterDataWebApp
  ports:
    - "3000:3000"
  depends_on:
    - mysqldb
    - mongodb
    - authentication
```

Figure 2: Enter Data Container

Upon accessing the web application, users are required to enter their login credentials via a secure login page. These credentials are then sent to the authentication service running on port 3001 to verify the user's authorization status. If the credentials are correct, the user is directed to the Enter Age page. However, if the credentials are incorrect, the user is redirected to the login page to re-enter their credentials.

Once the user is authorized, they can input their age on the Enter Age page, which is then securely transmitted to port 3306 (MySQL DB) for storage in the centralized database.

The Enter Data Web Application is developed using Node JS, a popular and versatile runtime environment for building scalable and efficient server-side applications. This ensures that the application is robust and reliable, with the ability to handle high volumes of traffic and data inputs.

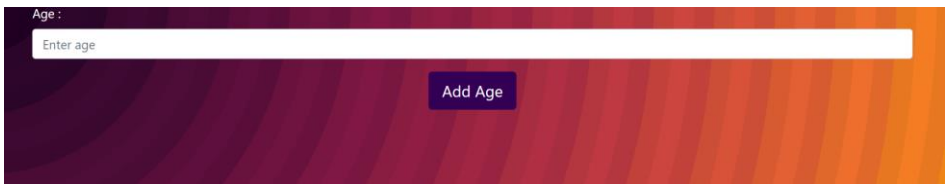


Figure 3: Screenshot of Enter Data web app.

4. Authentication Service

The Authentication Service is a crucial component of our system that ensures the security and privacy of user data. Running on port 3001, the service is responsible for validating user credentials, which are transmitted from the Enter Data Web Application.

The Authentication Service employs a static validation process to verify user credentials, using a combination of email and password as the primary validation parameters. Once the credentials are validated, the service returns a response to the Enter Data Web Application, indicating whether the user is authorized to access the system or not.

The Authentication Service is developed using Node JS.
Overall, the Authentication Service plays a critical role in our system, providing a secure and reliable means of authenticating user credentials and protecting user data.

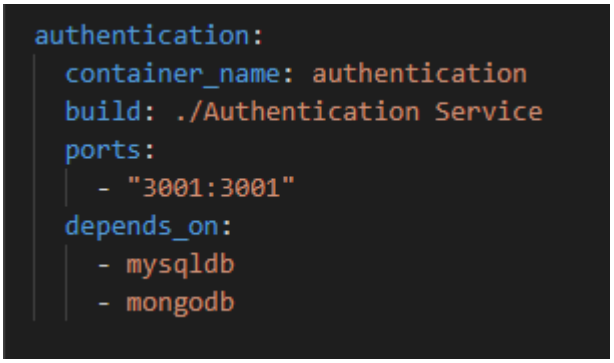


Figure 4: Authentication Container

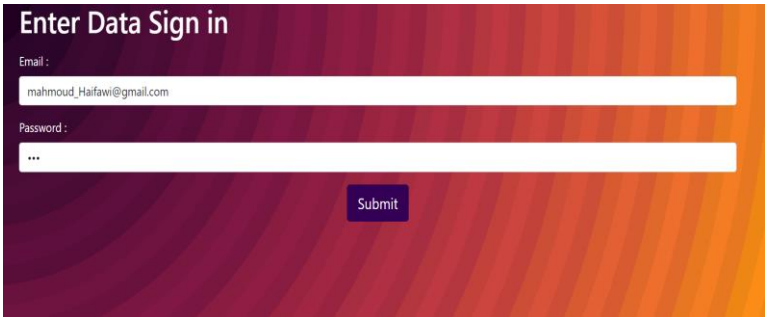


Figure 5: Login

5. MySQL Database

The MySQL container in question is not currently mapped to any port, however, it is connected to other containers within the default network of the Docker Compose environment. Access to this database by other containers within the same network is possible through the utilization of port 3306.

Within the MySQL container, there exists a single table titled Ages, which includes two columns: id and age. The id column is designed to function as a unique identifier for each row contained within the table, while the age column stores the corresponding age value for each row.

Interacting with this MySQL database is achievable through the utilization of a MySQL client or application, thereby enabling the execution of SQL commands for accessing and manipulating the data contained within the Ages table, as well as any other tables or databases present within the container.

```yaml
mysqldb:
  container_name: mysqldb
  image: mysql:8.0
  restart: unless-stopped
  volumes:
    - ./MySql DB/db:/var/lib/mysql
  environment:
    - MYSQL_HOST=localhost
    - MYSQL_PASSWORD=123456
    - MYSQL_ROOT_PASSWORD=root
```

Figure 6: MySQL container

6. Analytics Service

The service under consideration facilitates the analysis of data stored within a MySQL database. It is designed to be triggered by the EnterDataWabApp application, which operates through port 3000 and is responsible for making changes to the MySQL container.

To access the service, it is necessary to connect to port 3003, where the service is hosted. It is important to note that this application is built using the Node JS framework, which provides a robust and efficient platform for building scalable and high-performance applications.

```
analytics:
  container_name: analytics
  build: ./Analytics Service
  ports:
    - "3003:3003"
  depends_on:
    - mysqldb
    - mongodb
```

Figure 7: Analytics Container

7. MongoDB

The MongoDB container present in the Docker Compose environment is not currently mapped to any port accessible by the host. However, other containers within the same network can gain access to the database through port 27017.

Within this MongoDB container, there is a single collection known as age avg. Each document within this collection consists of two attributes: id and avg. These attributes likely function as a unique identifier and an average value, respectively, for each corresponding document within the collection.

The ability to interact with this MongoDB container is possible through

the utilization of a MongoDB client or application, which provides a means of accessing and manipulating data within the age avg collection, as well as any other collections or databases present within the container.

```
mongodb:
  container_name: mongodb
  image: mongo
  restart: always
  volumes:
    - ./Mongo DB/data:/data/db
  logging:
    driver: none
  command: mongod --quiet --logpath /dev/null
```

Figure 8: mongo DB Container

8. Show Results Web Application

The Show Results Web Application is designed to display the results generated by the analytics service hosted on port 3003.

Prior to viewing the results, users must provide their credentials to gain access. These credentials are then forwarded to an authentication service running on port 3001 to verify the user's authorization status. Once the user has been confirmed as authorized, they will be directed to a dedicated page to view the results.
The data for these results is retrieved from a MongoDB container running on port 27017.

The Show Results Web Application is mapped to port 3002, which provides a means of accessing the application's interface. This application was developed using the Node JS framework.

```
showresults:
  container_name: showresults
  build: ./ShowResultsWebApp
  ports:
    - "3002:3002"
  depends_on:
    - mysqldb
    - mongodb
    - authentication
```

Figure 9: Show Result container.

Figure 9: ShowResult Web applications

9. Mongo Express

Mongo Express is a lightweight, web-based administrative tool that facilitates the efficient management of MongoDB databases. Developed using the Node JS, Express, and Bootstrap3 frameworks, this tool simplifies several MongoDB administrative tasks. With Mongo Express, users can easily add, delete, or modify databases, collections, and documents as needed.

In your specific case, you utilized Mongo Express to conveniently view the contents of your MongoDB database. The container hosting Mongo Express was mapped to port 8082, which provides a means of accessing the tool's user interface. By utilizing Mongo Express, you were able to effectively manage your MongoDB databases and simplify several related administrative tasks.

```
mongo-express:
  image: mongo-express:1.0.0-alpha.4
  restart: unless-stopped
  ports:
    - 8082:8082
  environment:
    ME_CONFIG_MONGODB_SERVER: mongodb
  depends_on:
    - mysqldb
    - mongodb
  logging:
    driver: none
```

10. Docker Compose

docker-compose.yml

Project > docker-compose.yml

```yaml
version: '3'
services:

  enterdata:
    container_name: enterdata
    build: ./EnterDataWebApp
    ports:
      - "3000:3000"
    depends_on:
      - mysqldb
      - mongodb
      - authentication

  authentication:
    container_name: authentication
    build: ./Authentication Service
    ports:
      - "3001:3001"
    depends_on:
      - mysqldb
      - mongodb

  mysqldb:
    container_name: mysqldb
    image: mysql:8.0
    restart: unless-stopped
    volumes:
      - ./MySql DB/db:/var/lib/mysql
    environment:
      - MYSQL_HOST=localhost
      - MYSQL_PASSWORD=123456
      - MYSQL_ROOT_PASSWORD=root

  showresults:
    container_name: showresults
    build: ./ShowResultsWebApp
    ports:
      - "3002:3002"
    depends_on:
      - mysqldb
      - mongodb
      - authentication

  analytics:
    container_name: analytics
    build: ./Analytics Service
    ports:
      - "3003:3003"
    depends_on:
      - mysqldb
      - mongodb

  mongodb:
    container_name: mongodb
    image: mongo
    restart: always
    volumes:
      - ./Mongo DB/data:/data/db
    logging:
      driver: none
    command: mongod --quiet --logpath /dev/null

  mongo-express:
    image: mongo-express:1.0.0-alpha.4
    restart: unless-stopped
    ports:
      - 8082:8082
    environment:
      ME_CONFIG_MONGODB_SERVER: mongodb
    depends_on:
      - mysqldb
      - mongodb
    logging:
      driver: none
    #command to disable mongo express logging
    #command: mongod --quiet --logpath /dev/null

```

Name	Image	Status	Port(s)	Started	Actions
project	-	Running (7/7)			
mysqldb 8831dde431ca	mysql:8.0	Running		13 minutes ag	
mongodb 9511ced1244c	mongo	Running		13 minutes ag	
analytics 0daa4dee16c2	project-analytics	Running	3003:3003	13 minutes ag	
authentication 358b4c80dbc2	project-authentication	Running	3001:3001	13 minutes ag	
mongo-express-1 4f47903b84b0	mongo-express:1.0.0-alpha.4	Running	8082:8082	13 minutes ag	
showresults b3c1ef42308b	project-showresults	Running	3002:3002	13 minutes ag	
enterdata 8c8519586b21	project-enterdata	Running	3000:3000	13 minutes ag	